TECNOLÓGICO DE MONTERREY

Escuela de Ingeniería y Ciencias · Departamento de Ciencias Computacionales

PROYECTO INTEGRADOR · CORRIDA V4

Proponer el máximo a partir de N potencias medidas — bitácora razonada paso a paso (nivel 1)

Aprendizaje por refuerzo con MLP (REINFORCE y PPO) frente a referencias sin aprendizaje y al MSAPSO del paper

Implementación del esquema propuesto por el profesor: N potencias medidas como observación, política gaussiana y muestreo de puntos aledaños. Incluye el razonamiento de cada paso, los resultados, los hallazgos, las limitaciones y los siguientes pasos.

Sponsor: Prof. Gerardo Jesús Camacho González, PhD. · Monterrey, Nuevo León, México · Octubre de 2026

Índice

1. Resumen ejecutivo 3

2. Paso a paso de la corrida V4 5

Paso 1 — Diseño: traducir las indicaciones del profesor a un problema bien definido 6

Paso 2 — Simulador V4 y «planta estática» (nivel 1) 7

Paso 3 — Entorno Gymnasium y pruebas de humo 8

Paso 4 — Referencias sin aprendizaje en el nivel 1 (¿contra qué hay que ganar?) 9

Paso 5 — Cota de diagnóstico: ¿hay margen para un modelo? 11

Paso 6 — REINFORCE con MLP (propuesta del profesor) 12

Paso 7 — PPO (stable-baselines3) en el mismo entorno 13

Paso 8 — ¿Más entrenamiento cambia el resultado? 14

3. Conclusiones 15

4. Siguientes pasos y preguntas para el profesor 16

5. Limitaciones y supuestos declarados 17

6. Mapa de archivos y cómo reproducir 18

Anexo. Código clave 19

01

1. Resumen ejecutivo

Esta corrida (V4) implementa el esquema que propuso el profesor el 21-oct-2026: medir N potencias en N corrientes uniformes y ascendentes, y que un agente de RL con un MLP sencillo las use como observación para proponer dónde está el máximo; el muestreo de puntos aledaños es parte de la política gaussiana. Respetamos sus indicaciones: sin GMPP real en entrenamiento, rango fijado con la Isc nominal y métrica que prioriza el número de mediciones. Esta etapa cubre el **nivel 1** (potencia estática, sin dinámica del convertidor) con N = 7 y K = 5.

> **Respuesta corta.** Con esta formulación, REINFORCE-MLP y PPO **no superan** a una referencia sin aprendizaje («mejor de los 7 puntos + 5 muestras gaussianas»), que ya usa solo 12 mediciones en lugar de las 175 del MSAPSO. Un modelo supervisado, usado solo como cota de diagnóstico, indica que sí hay margen (≈ +0.9 pp de exactitud y ≈ +0.8 pp de energía sobre la referencia, en sintéticos), pero el RL no lo alcanza, y 15 veces más entrenamiento no cambia el resultado. La causa probable es el objetivo (potencia promedio de las muestras), no la falta de entrenamiento.

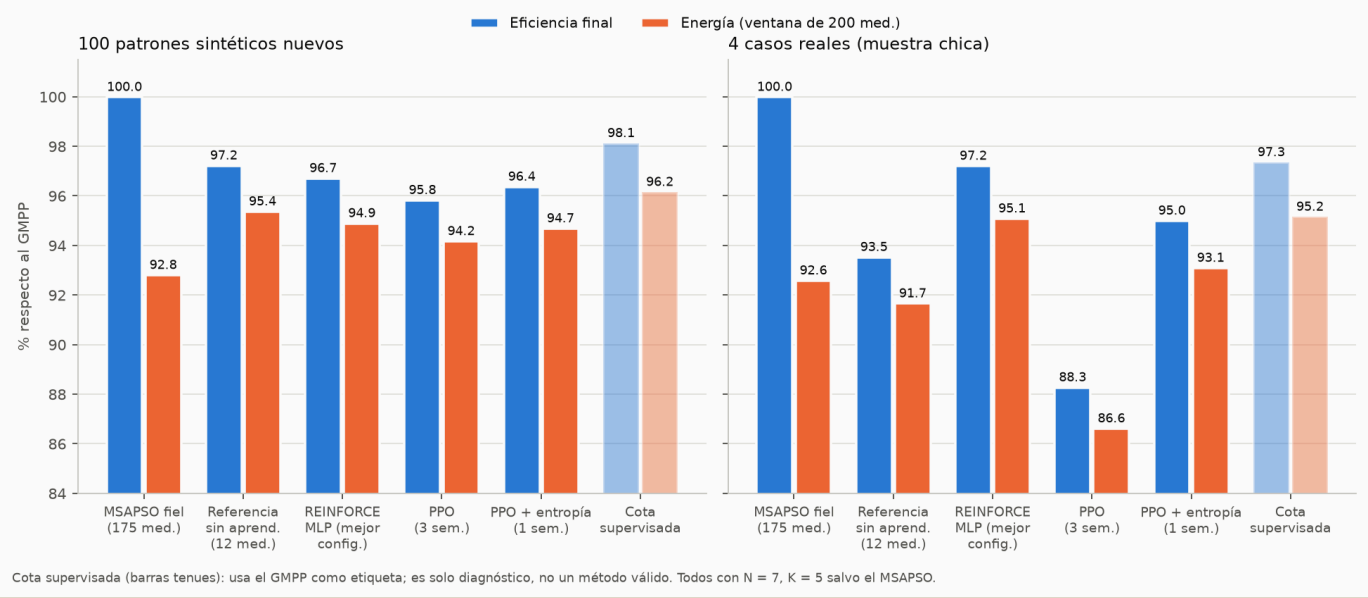


Figura A. Comparación en 100 patrones sintéticos nuevos y en los 4 casos reales (N = 7, K = 5; 12 mediciones salvo el MSAPSO, que usa 175). Eficiencia final = potencia en el punto elegido / GMPP; energía = promedio en una ventana de 200 mediciones. Las barras tenues son una cota de diagnóstico que usa el GMPP como etiqueta.

Hallazgos principales

1. **Referencia a vencer.** «Mejor de 7 + 5 gaussianas» logra 97.2 % de exactitud final y 95.4 % de energía en 100 sintéticos con 12 mediciones; el MSAPSO fiel logra 99.98 % y 92.8 % con 175 (Paso 4).

2. **Hay margen, pero moderado.** La cota supervisada (no es un método válido: usa el GMPP como etiqueta) llega a 98.1 % y 96.2 % (Paso 5). Ningún modelo con 12 mediciones iguala la exactitud del MSAPSO.
3. **REINFORCE-MLP:** 96.7–96.9 % / 94.9–95.0 % en sintéticos, ligeramente por debajo de la referencia; en los 4 reales parece mejor (+3 pp) pero es una muestra chica. Su media μ solo llega a ≈ 94.4 %, igual que «mejor de 7» (Paso 6).
4. **PPO:** 95.6–96.2 % / 93.9–94.5 % en sintéticos y 88–89 % en reales. La σ colapsa a 0.013; con más entropía mejora en reales (95 %) (Paso 7).
5. **Más entrenamiento no ayuda:** 15 veces más actualizaciones suben el resultado ≈ 0.4 pp y solo igualan a la referencia (Paso 8).
6. **Efecto de la Isc nominal:** sembrar el MSAPSO con la Isc nominal lo empeora (energía 91.0 y 89.9 % frente a 92.8 y 92.6 %) porque en los casos 1 y 3 la Isc real es menor (Paso 4).

Qué sigue (propuesto)

Corregir la raíz antes de seguir con más algoritmos: premiar el *mejor* de las K muestras y no el promedio, mantener σ fija o decreciente, entrenar con más muestras que en operación, y evaluar una etiqueta por barrido denso *medido*. Después, SAC, DDPG y árbol de decisión sobre la formulación corregida, y el barrido de N y K. Detalle en la sección 4.

2. Paso a paso de la corrida V4

Cada paso indica **qué se hizo**, **por qué** (el razonamiento), cómo se **verificó**, el **resultado** y los **archivos** que lo respaldan.

Documento vivo: cada entrada registra QUÉ se hizo, POR QUÉ (razonamiento), cómo se VERIFICÓ, QUÉ RESULTÓ y QUÉ ARCHIVOS lo respaldan. De aquí se compila el PDF.

Contexto y punto de partida (octubre 2026)

- La V2 (carpeta `Proyecto Integrador Paneles_V2/`) NO se modifica. Concluyó que el agente PPO que ajustaba β y el reinicio solo mejoraba +1.84 pp en sombra estática y que una regla simple sin RL lo igualaba.
- En la reunión del 21-oct-2026 el profesor propuso otro esquema: medir N potencias en N corrientes uniformes y ascendentes, y que un agente de RL con un MLP sencillo las use como observación para proponer dónde está el máximo; el muestreo de puntos aledaños es parte de policy gradients (política gaussiana con σ pequeña). Orden de algoritmos: REINFORCE con baselines, PPO, SAC y DDPG; variante con árbol de decisión dentro del RL.
- Indicaciones posteriores del profesor: no usar el GMPP real en entrenamiento (evitar fuga de datos); rango de corrientes fijado con la Isc nominal; t_wait del paper como base y preferir iteraciones al tiempo (su Colab no modela la dinámica del convertidor); medir número de mediciones y energía, priorizando mediciones; reinicio ante 5 W remuestreando los N puntos (aprender el umbral queda para después); incluir sombra gradual y ruido de medición.
- Alcance de esta corrida (V4): el NIVEL 1 (potencia estática, sin dinámica del convertidor) con N = 7 y K = 5. Quedan para la siguiente corrida: SAC, DDPG, árbol de decisión, barrido de N y K, sombra gradual, ruido en entrenamiento y el nivel dinámico.

PASO 1

Paso 1 — Diseño: traducir las indicaciones del profesor a un problema bien definido

Qué hice. Escribí `diseno_V4.md` con las indicaciones del profesor, las decisiones de diseño y el estado. Observación = N potencias medidas / 250 W (potencia nominal de hoja de datos) en N corrientes uniformes entre 0.10 y 0.95 de la Isc NOMINAL (8.9 A). Acción = corriente propuesta en ese rango. Recompensa = potencia MEDIDA en la corriente muestreada / 250 W. Episodio = N + K mediciones (K muestras por patrón); es un bandido contextual ($\gamma = 0$).

Por qué. (a) El profesor pidió explícitamente no usar el GMPP real en entrenamiento: la observación y la recompensa solo pueden usar potencias medidas; el GMPP se reserva para calcular eficiencias al EVALUAR. (b) Pidió fijar mínimo y máximo con la Isc nominal, no con la real del patrón (que un controlador no conoce). (c) Su Colab ignora la dinámica del convertidor, así que la métrica prioritaria es el número de mediciones.

Decisiones propias (por confirmar con el profesor). Normalizar por 250 W; episodios de K muestras con $\gamma = 0$; dos niveles de simulación (1: estático, 2: con convertidor); referencias sin aprendizaje; la ventana de 200 mediciones para la métrica de energía.

Archivos. `diseno_V4.md` .

PASO 2

Paso 2 — Simulador V4 y «planta estática» (nivel 1)

Qué hice. Copié el simulador de la V2 a `pvsim_V4/` (mismos modelos del panel, convertidor y MSAPSO fiel, renombrados a `_V4`) y añadí `static_plant_V4.py` con dos plantas: `StaticPlant` (nivel 1: potencia estática P(I), sin inercia del inductor) y `DynamicPlantLogged` (nivel 2: simulador con Boost + Super-Twisting, para usarlo después). Ambas registran cada medición y admiten ruido gaussiano aditivo en la potencia leída. `medicion_V4.py` define la rejilla, la observación y las métricas.

Por qué. El profesor indicó que conviene contar iteraciones y no tiempo, porque el tiempo de transición depende de muchas cosas. Con la planta estática, la potencia medida al pedir I es P(I) de la curva; una corriente mayor que la Isc REAL del patrón da potencia 0 (el panel queda en cortocircuito), lo que importa porque el rango se fija con la Isc nominal y los casos 1 y 3 tienen Isc real menor (7.12 A y 3.25 A frente a 8.9 A).

Métricas (por patrón, búsqueda en frío). `n_meas` (mediciones gastadas), `n99` (primera medición con potencia verdadera ≥ 99 % del GMPP), `eff_final` (potencia verdadera en el punto elegido, que es la mejor potencia LEÍDA, entre el GMPP) y `eff_energy` (promedio de potencia verdadera en una ventana de 200 mediciones: la búsqueda más la permanencia en el punto elegido).

Advertencia declarada. `eff_energy` depende de la longitud de la ventana (200) y favorece búsquedas cortas; por eso se reportan también `eff_final` y `n_meas`.

Archivos. `pvsim_V4/static_plant_V4.py`, `pvsim_V4/medicion_V4.py`.

PASO 3

Paso 3 — Entorno Gymnasium y pruebas de humo

Qué hice. `pvsim_V4/gym_env_V4.py` : un episodio es un patrón sintético al azar (4 sub-strings); el entorno mide N potencias ascendentes (observación) y el agente da K acciones; cada una es una corriente en [I_MIN, I_MAX] y la recompensa es la potencia medida entre 250 W. Al terminar entrega las métricas en `info['metrics']` .

Verificación. `probar_entorno_V4.py` : el entorno pasa `check_env` de stable-baselines3 en nivel estático y dinámico (0.5 y 5 ms). Un agente que propone el GMPP real obtiene 100 % en el nivel estático (el problema está bien planteado) y uno al azar 95.8 % (porque el mejor de los N + K puntos ya incluye el barrido).

Nota. En el nivel dinámico la eficiencia aún se calcula contra el GMPP teórico y debería calcularse contra P_ceiling (el Boost con R = 12 Ω no alcanza todos los puntos); queda pendiente antes de usar ese nivel.

Archivos. `pvsim_V4/gym_env_V4.py` , `probar_entorno_V4.py` .

PASO 4

Paso 4 — Referencias sin aprendizaje en el nivel 1 (¿contra qué hay que ganar?)

Qué hice. `baselines_nivel1_V4.py` mide, con el mismo protocolo, métodos sin aprendizaje sobre los 4 casos reales y 100 patrones sintéticos NUEVOS (semilla 2026, distintos de los de entrenamiento), con ruido de 0, 1 y 2 W: MSAPSO fiel (Fig. 6 del paper: 7 partículas, β 0.1→0.95, t_max = 25 = 175 mediciones), sembrado con la Isc real y con la Isc nominal; la regla simple de la V2 (β = 0.95, t_max = 5 = 35 mediciones); «mejor de N» (N puntos uniformes y se elige el mejor); y «mejor de N + 5 gaussianas» (los N puntos y luego K = 5 muestras N(mejor, σ^2) con σ = 0.05·Isc_nom; es lo que haría la política SIN haber aprendido nada).

Por qué. Antes de entrenar hay que saber qué margen deja el control sin aprendizaje; si un método trivial ya iguala al RL, el RL no aporta.

Resultado (sin ruido; sintéticos / reales).

MÉTODO	MEDICIONES	EFICIENCIA FINAL	ENERGÍA (200 MED.)
MSAPSO fiel (Isc real)	175	99.98 / 99.99 %	92.8 / 92.6 %
MSAPSO fiel (Isc nominal)	175	99.97 / 99.99 %	91.0 / 89.9 %
Regla simple (β = 0.95, t_max = 5)	35	94.8 / 94.4 %	93.1 / 92.5 %
Mejor de 7	7	93.3 / 87.1 %	91.9 / 85.6 %
Mejor de 7 + 5 gaussianas	12	97.2 / 93.5 %	95.4 / 91.7 %
Mejor de 10 + 5 gaussianas	15	98.4 / 96.5 %	95.9 / 93.9 %
Mejor de 20	20	98.5 / 97.2 %	94.3 / 92.5 %
Mejor de 20 + 5 gaussianas	25	99.2 / 99.6 %	94.6 / 94.4 %

Hallazgos. (1) Sembrar el MSAPSO con la Isc nominal lo empeora (energía 91.0 y 89.9 % frente a 92.8 y 92.6 %), porque en los casos 1 y 3 la Isc real es menor y varias partículas caen donde casi no hay potencia: es una consecuencia de la indicación del profesor que conviene comunicarle. (2) «Mejor de 7 + 5 gaussianas» usa 12 mediciones y ya supera al MSAPSO en energía en sintéticos (95.4 contra 92.8 %), aunque pierde exactitud final (97.2 contra casi 100 %). (3) Con ruido de 1 y 2 W las cifras cambian poco en este nivel.

Archivos. `baselines_nivel1_V4.py` , `resultados_V4/baselines_nivel1_V4.txt|json` .

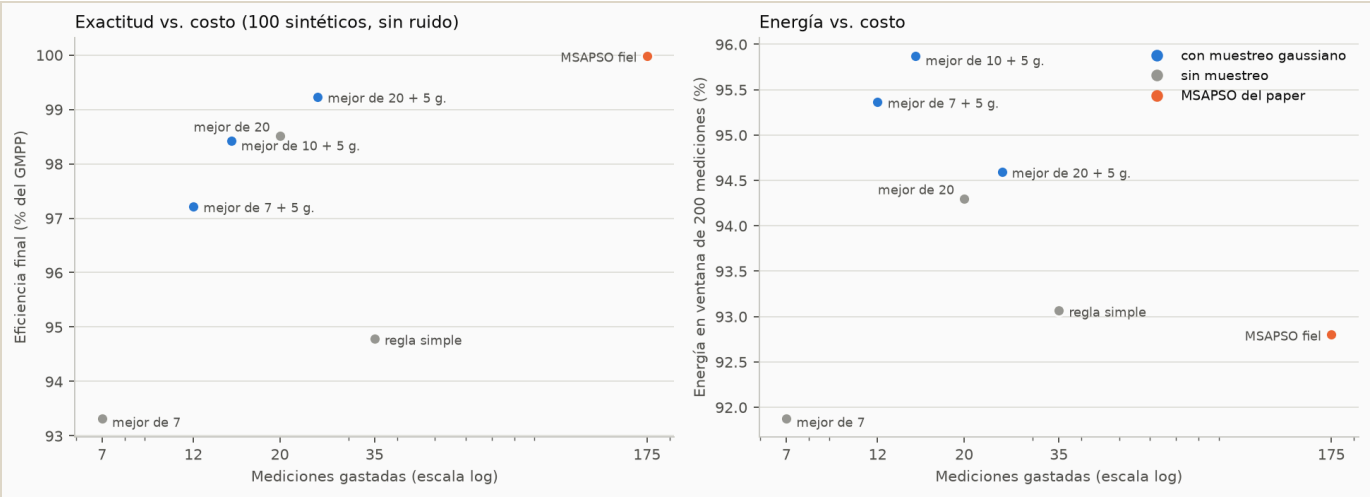


Figura B. Costo (mediciones, escala logarítmica) contra exactitud final y energía, en 100 patrones sintéticos nuevos sin ruido. Azul = con muestreo gaussiano; gris = sin muestreo; naranja = MSAPSO fiel del paper.

PASO 5

Paso 5 — Cota de diagnóstico: ¿hay margen para un modelo?

Qué hice. `cota_diagnostico_V4.py` entrena un MLP SUPERVISADO (con el GMPP real como etiqueta) sobre 6000 curvas sintéticas, con las mismas N potencias, y evalúa con el mismo protocolo. También calcula un «oráculo» que propone exactamente el GMPP y muestrea K = 5 gaussianas alrededor.

Por qué. Para saber, antes de gastar entrenamiento de RL, cuánto margen existe. NO es un método de la V4: el profesor prohíbe el GMPP real en entrenamiento; es un diagnóstico del techo.

Resultado (N = 7, K = 5, 12 mediciones). Cota supervisada: sintéticos 98.1 % final / 96.2 % energía; reales 97.4 / 95.2 %. Con N = 10 (15 mediciones): 98.1 / 95.5 % (sintéticos); con N = 20 (25 mediciones): 99.0 / 94.3 %. Incluso el oráculo con σ = 0.05 queda en 96–99 % porque 5 muestras no siempre caen en el pico, lo que indica que σ y K también limitan el techo.

Lectura. Hay margen moderado: frente a la referencia sin aprendizaje (97.2 / 95.4 % en sintéticos) la cota sube ≈ 0.9 pp de exactitud y ≈ 0.8 pp de energía; frente al MSAPSO, ≈ +3.4 pp de energía con 12 mediciones en vez de 175. Ningún modelo con 12 mediciones alcanza la exactitud del MSAPSO.

Archivos. `cota_diagnostico_V4.py` , `resultados_V4/cota_diagnostico_V4.txt` .

PASO 6

Paso 6 — REINFORCE con MLP (propuesta del profesor)

Qué hice. `reinforce_nn_V4.py` : política MLP(N → 64 → 64 → 1, tanh) que da μ (corriente propuesta normalizada); se muestrean K = 5 acciones $a_k \sim N(\mu, \sigma^2)$ por patrón; recompensa r_k = potencia MEDIDA / 250 W; gradiente REINFORCE $\nabla J = E[(r_k - b)\cdot(a_k - \mu)/\sigma^2 \cdot \nabla\mu]$. Baselines b: ninguno, «mean» (promedio de las otras K − 1 muestras del mismo patrón) y «value» (V(obs) aprendida). Lotes de 128 patrones, Adam con lr 1e-3, 4000 actualizaciones, banco de 6000 curvas sintéticas de entrenamiento (semilla 0). Sin GMPP ni P_ceiling en el entrenamiento. Evaluación en los 4 reales y 100 sintéticos nuevos, con 3 semillas, en tres modos: K muestras, solo μ (1 medición) y μ + K muestras. Malla: baseline × $\sigma \in \{0.02, 0.05, 0.10\}\cdot$Isc_nom (`grid_reinforce_V4.py`).

Resultado (media de 3 semillas, modo K muestras, N = 7, K = 5).

BASELINE, Σ	REALES: FINAL / ENERGÍA	SINTÉTICOS: FINAL / ENERGÍA
none, 0.05 (mejor)	97.2 / 95.1 %	96.7 / 94.9 %
mean, 0.05	96.3 / 94.2 %	96.8 / 95.0 %
value, 0.05	95.6 / 93.5 %	96.7 / 94.9 %
none, 0.02	96.4 / 94.5 %	96.1 / 94.4 %
none, 0.10	95.3 / 93.1 %	96.9 / 94.9 %
Evaluado solo en μ (1 medición): ≈ 94.4 % en sintéticos, apenas sobre «mejor de 7» (93.3 %).		

Variante de recompensa. Probé premiar la mejora sobre el mejor punto del barrido (calculada con potencias medidas): no cambió nada (`prueba_reward_V4.py`).

Lectura. En sintéticos REINFORCE queda ligeramente por debajo de la referencia sin aprendizaje (97.2 / 95.4 %). En los 4 reales parece mejor (+3 pp) pero son solo 4 casos. La política casi no aprende dónde está el pico: su media μ equivale a «mejor de 7».

Archivos. `reinforce_nn_V4.py` , `grid_reinforce_V4.py` , `prueba_reward_V4.py` , `resultados_V4/reinforce_*` .

PASO 7

Paso 7 — PPO (stable-baselines3) en el mismo entorno

Qué hice. `ppo_nivel1_V4.py` : PPO sobre `MPPTSampleEnvV4` con γ = 0 y λ_GAE = 0 (bandido contextual), MLP 64×64, 8 entornos, 1 millón de pasos (≈ 85 s por semilla), σ inicial pequeña (log_std_init = −2, σ = 0.135 en unidades de acción), 3 semillas. Mismos escenarios de evaluación.

Resultado.

CONFIGURACIÓN	REALES: FINAL / ENERGÍA	SINTÉTICOS: FINAL / ENERGÍA	Σ APRENDIDA
PPO, semilla 0	87.9 / 86.3 %	96.2 / 94.5 %	0.014
PPO, semilla 1	87.9 / 86.2 %	95.7 / 94.1 %	0.013
PPO, semilla 2	89.0 / 87.4 %	95.6 / 93.9 %	0.013
PPO + entropía 0.02 (1 sem.)	87.8 / 86.1 %	95.8 / 94.2 %	0.019
PPO + entropía 0.1 (1 sem.)	95.0 / 93.1 %	96.4 / 94.7 %	0.034

Hallazgo. La σ colapsa a ≈ 0.013 (frente a 0.135 inicial): al maximizar la potencia promedio de las muestras, la política aprende a no explorar, con lo que las K muestras caen casi en el mismo punto y no aportan nada. Con una bonificación de entropía alta (0.1), PPO mejora mucho en los reales (88 → 95 %), lo que apunta en esa dirección. Es una hipótesis razonable, no una causa demostrada del todo.

Archivos. `ppo_nivel1_V4.py` , `resultados_V4/ppo_*` , `modelos_V4/` .

PASO 8

Paso 8 — ¿Más entrenamiento cambia el resultado?

Qué hice. `entrenamiento_largo_V4.py` : REINFORCE (baseline «mean», σ = 0.05) con 4 mil, 20 mil y 60 mil actualizaciones y lotes de 128, 512 y 256, 3 semillas.

Resultado (sintéticos, eficiencia final con K = 5 / solo μ). 4 mil: 96.84 / 94.36 %; 20 mil (lote 128): 97.10 / 94.52 %; 20 mil (lote 512): 96.97 / 94.48 %; 60 mil: 97.20 / 94.76 %. Referencia sin aprendizaje: 97.22 %. Cota supervisada: 98.1 %.

Lectura. Con 15 veces más entrenamiento el resultado sube ≈ 0.4 pp y se IGUALA a la referencia sin aprendizaje, sin superarla; μ solo sube de 94.4 a 94.8 %. La curva se aplana, lo que apunta a que el límite está en lo que se optimiza y no en la cantidad de entrenamiento. En la V2 ocurrió algo parecido: PPO con 3 M de pasos fue peor que con 1 M.

Archivos. `entrenamiento_largo_V4.py` , `resultados_V4/entrenamiento_largo_V4.txt` .

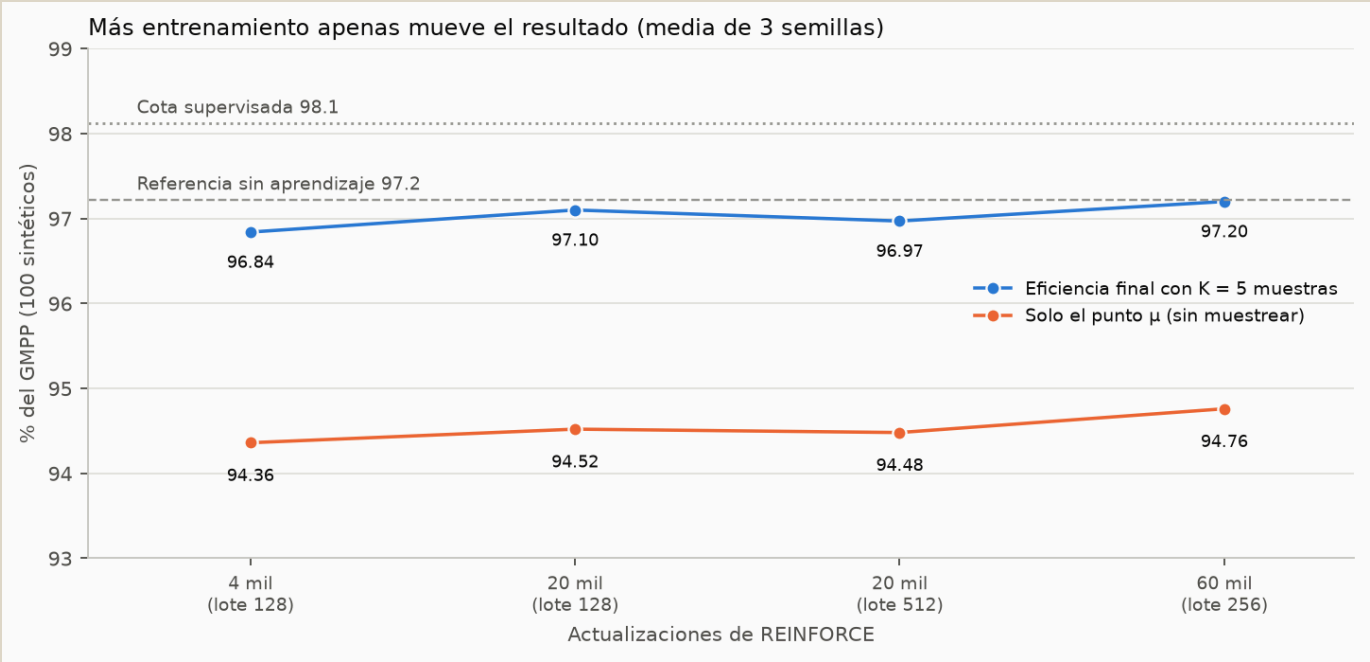


Figura C. REINFORCE con 15 veces más entrenamiento (media de 3 semillas): la eficiencia final apenas sube y solo iguala a la referencia sin aprendizaje; el punto μ casi no mejora.

3. Conclusiones

Qué se puede afirmar

- El esquema propuesto por el profesor está bien planteado y es implementable: el entorno es válido y un agente que propone el GMPP real obtiene 100 % en el nivel estático.
- Con 12 mediciones (7 de barrido + 5 muestras) un método **sin aprendizaje** ya iguala o supera al MSAPSO fiel en energía en sintéticos (95.4 contra 92.8 %), a costa de exactitud final (97.2 contra casi 100 %).
- Con la formulación actual, **REINFORCE-MLP y PPO no mejoran a esa referencia**; con más entrenamiento solo la igualan (60 mil actualizaciones de REINFORCE: 97.20 frente a 97.22 %).

Qué no se puede afirmar

- Que el RL nunca ayudará: no se probaron SAC, DDPG, el árbol de decisión, otros N y K, ni la formulación corregida.
- Que REINFORCE sea mejor que la referencia en reales: son solo 4 casos.
- La causa del colapso de σ está sustentada por una prueba corta (PPO con entropía, 1 semilla); es una hipótesis razonable, no una demostración.

> **Para el profesor.** Si el criterio es «no perder exactitud», este esquema no supera al MSAPSO (el techo con 12 mediciones es ≈ 97–98 %, contra casi 100 %). Si el criterio es «menos mediciones», el esquema reduce el costo unas 15 veces con ≈ 2 pp menos de exactitud final. Hay que acordar cuál criterio vale.

4. Siguientes pasos y preguntas para el profesor

Propuesta de la corrida V5 (no ejecutada en esta entrega). Se corrige primero la raíz y solo después se comparan más algoritmos, para que cada resultado responda una sola pregunta.

1. **Correcciones de raíz con REINFORCE y PPO (rápido).** (a) Premiar el *mejor* de las K muestras, no el promedio, que es lo que se usa al elegir el punto. (b) σ fija o decreciente (o más entropía) para que no colapse. (c) Entrenar con más muestras que en operación (K = 20 al entrenar, K = 5 al evaluar). Si no acercan los resultados a la cota, el problema es más profundo y hay que replantear el esquema.
2. **Si mejoran:** SAC, DDPG y árbol de decisión sobre la formulación corregida, comparados en igualdad de condiciones.
3. **Barrer N (7, 10, 20) y K.**
4. **Después:** reinicio ante 5 W, sombra gradual, ruido de medición en entrenamiento, nivel dinámico (corrigiendo la eficiencia contra P_ceiling), optimización bayesiana como referencia.

Preguntas para el profesor

1. ¿Acepta ≈ 2 pp menos de exactitud final a cambio de unas 15 veces menos mediciones? Si no, esta vía no supera al MSAPSO en exactitud.
2. La energía se mide en una ventana de 200 mediciones, que favorece búsquedas cortas: ¿le parece una medida razonable?
3. ¿Una etiqueta basada en un barrido denso *medido* durante el entrenamiento respeta la regla de no usar el GMPP real?
4. Sembrar el MSAPSO con la Isc nominal lo empeora (casos 1 y 3): ¿lo comparamos así o con la Isc real?

5. Limitaciones y supuestos declarados

- **Nivel 1 solamente** (potencia estática): no incluye la dinámica del convertidor, el reinicio por 5 W, la sombra gradual ni el ruido en entrenamiento (el ruido solo se evaluó en las referencias).
- **N = 7 y K = 5** ($\sigma = 0.05 \cdot I_{sc_nom}$): sin barrido de N, K ni σ más allá de la malla de REINFORCE.
- **Solo 4 casos reales:** las cifras en reales son frágiles; las de sintéticos (100 patrones nuevos, semilla 2026) son más estables.
- **Pocas semillas:** 3 en REINFORCE y PPO; 1 en PPO con entropía.
- **Hiperparámetros por defecto o poco explorados:** sin búsqueda sistemática de lr, tamaño de red o tamaño de lote en PPO.
- **La métrica de energía** (ventana de 200 mediciones) favorece búsquedas cortas; deben leerse junto a la exactitud final y el número de mediciones.
- **La cota supervisada** usa el GMPP como etiqueta: es diagnóstico, no un método válido bajo la regla del profesor.
- **Supuestos propios por confirmar:** normalizar por 250 W, $\gamma = 0$, la ventana de 200 mediciones y el rango $[0.10, 0.95] \cdot I_{sc_nom}$.
- **Pendientes de la V4:** SAC, DDPG, árbol de decisión, nivel dinámico (con eficiencia contra $P_{ceiling}$) y optimización bayesiana.

6. Mapa de archivos y cómo reproducir

Mapa de archivos

ARCHIVO	QUÉ CONTIENE / QUÉ PRODUCE
`diseno_V4.md` , `bitacora_V4.md`	Diseño con las indicaciones del profesor y registro paso a paso (fuente de este PDF).
`pvsim_V4/`	Simulador (curvas, convertidor con numba, MSAPSO fiel), `static_plant_V4.py` (nivel 1 y nivel 2), `medicion_V4.py` (rejilla, observación, métricas) y `gym_env_V4.py` (entorno Gymnasium).
`datasets_V4/`	Los 4 casos reales del paper.
`probar_entorno_V4.py`	Pruebas de humo del entorno (Paso 3).
`baselines_nivel1_V4.py` → `resultados_V4/baselines_nivel1_V4.*`	Referencias sin aprendizaje (Paso 4).
`cota_diagnostico_V4.py` → `resultados_V4/cota_diagnostico_V4.txt`	Cota supervisada de diagnóstico (Paso 5).
`reinforce_nn_V4.py` , `grid_reinforce_V4.py` , `prueba_reward_V4.py` → `resultados_V4/reinforce_*`	REINFORCE con MLP, malla y variante de recompensa (Paso 6).
`ppo_nivel1_V4.py` → `resultados_V4/ppo_*` , `modelos_V4/`	PPO y modelos entrenados (Paso 7).
`entrenamiento_largo_V4.py` → `resultados_V4/entrenamiento_largo_V4.txt`	Efecto de entrenar más (Paso 8).
`graficas_V4.py` , `construir_reporte_V4.py`	Figuras y compilación de este PDF.

Cómo reproducir (con `requirements_V4.txt` , Python 3.10 o superior)

```
python3 probar_entorno_V4.py            # Paso 3: entorno valido
python3 baselines_nivel1_V4.py          # Paso 4: referencias sin aprendizaje
python3 cota_diagnostico_V4.py          # Paso 5: cota supervisada (diagnostico)
python3 grid_reinforce_V4.py            # Paso 6: malla de REINFORCE (unos minutos)
for s in 0 1 2; do python3 ppo_nivel1_V4.py --N 7 --K 5 --steps 1000000 --seed $s; done   # Paso 7 (~85 s c/u)
python3 entrenamiento_largo_V4.py       # Paso 8
python3 graficas_V4.py                  # figuras
python3 construir_reporte_V4.py         # PDF (requiere Google Chrome en macOS y pdftotext)
```

Todo corre en CPU. Los entrenamientos del nivel 1 tardan segundos o pocos minutos, no horas.

Anexo. Código clave

pvsim_V4/medicion_V4.py

57 LÍNEAS

```
"""
medicion_V4 — Definicion unica de la rejilla de medicion, la observacion y las metricas del NIVEL 1 (algoritmico) de la V4.

Decisiones de diseno (cada una responde a una indicacion del profesor; ver diseno_V4.md):
  * Rango de corrientes [I_MIN, I_MAX] = [0.10, 0.95] * ISC_NOMINAL (Isc de la hoja de datos, 8.9 A): «Isc nominal para fijar
    minimo y maximo». NO se usa la Isc real del patron (un controlador real no la conoce). Mismas fracciones que el paper.
  * Observacion = las N potencias medidas en N corrientes UNIFORMES entre I_MIN e I_MAX, en orden ASCENDENTE (no perturba
    el circuito), divididas entre la potencia nominal del panel (250 W, dato de hoja de datos; no es el GMPP).
  * La recompensa usa SOLO potencias medidas. El GMPP real (P_gmpp) se usa unicamente para EVALUAR (calcular eficiencias).
Metricas (por patron de sombra, busqueda en frio):
  n_meas   : mediciones gastadas hasta fijar el punto (prioridad del profesor)
  n99      : numero de la primera medicion cuya potencia VERDADERA >= 99 % del GMPP (nan si nunca)
  eff_final: potencia verdadera en el punto elegido (la mejor LEIDA) / GMPP
  eff_energy: promedio de la potencia verdadera durante una ventana de W mediciones (busqueda + permanencia en el punto
              elegido) / GMPP. Es la «energia en unidades de iteracion»; mismo W para todos los metodos.
"""
import numpy as np
from pvsim_V4.array_model_V4 import ISC_REF

ISC_NOM = ISC_REF            # 8.9 A (hoja de datos)
FRAC_MIN, FRAC_MAX = 0.10, 0.95
I_MIN, I_MAX = FRAC_MIN * ISC_NOM, FRAC_MAX * ISC_NOM
P_NOM = 250.0                # W, potencia nominal del panel (hoja de datos)
W_ENERGIA = 200              # ventana (en mediciones) para la metrica de energia


def grid_currents(n):
    return np.linspace(I_MIN, I_MAX, n)


def scan(plant, n):
    """Mide la potencia en n corrientes uniformes, en orden ascendente. Devuelve (I, P_leida)."""
    I = grid_currents(n)
    P = np.array([plant.run(1.0, i)[0] for i in I])
    return I, P


def action_to_current(a):
    """a en [-1, 1] -> corriente en [I_MIN, I_MAX]."""
    return I_MIN + 0.5 * (float(np.clip(a, -1, 1)) + 1.0) * (I_MAX - I_MIN)


def current_to_action(I):
    return 2.0 * (np.clip(I, I_MIN, I_MAX) - I_MIN) / (I_MAX - I_MIN) - 1.0


def metrics_from_plant(plant, curve, window=W_ENERGIA):
    Pt, Pm, I = np.array(plant.log_Ptrue), np.array(plant.log_Pmeas), np.array(plant.log_I)
    Pg = curve.P_gmpp
    n = len(Pt)
    sel = int(np.where(Pm == Pm.max())[0][-1])          # el controlador elige la mejor potencia LEIDA (empates: la ultima)
    Psel = Pt[sel]
    ok = np.where(Pt >= 0.99 * Pg)[0]
    head = Pt[:window]
    energy = (head.sum() + max(window - n, 0) * Psel) / (window * Pg)
    return dict(n_meas=n, n99=float(ok[0] + 1) if len(ok) else float("nan"),
                eff_final=100.0 * Psel / Pg, eff_energy=100.0 * energy, I_sel=float(I[sel]))
```

pvsim_V4/static_plant_V4.py

75 LÍNEAS

```
"""
static_plant_V4 — «Planta» de NIVEL ALGORITMICO (nivel 1) para la V4.

El profesor indico que su Colab NO modela la dinamica del convertidor y que conviene medir la cantidad de ITERACIONES
(mediciones) y no el tiempo, porque el tiempo de transicion depende de muchas cosas. Esta clase hace exactamente eso:
al pedir una corriente I_ref la potencia medida es la ESTATICA de la curva, P(I_ref), sin inercia del inductor.
Tiene la misma interfaz que converter_V4.Plant (run / Ipv / set_curve) para reutilizar Swarm/MSAPSO sin cambios, y registra
cada medicion (cuantas se gastaron = la metrica principal).

  * Corriente pedida > Isc real del patron => el panel opera en cortocircuito: potencia 0 (importa porque el rango de
```

```
    busqueda se fija con la Isc NOMINAL de la hoja de datos, no con la real; con sombra la real es menor).
  * Ruido de medicion: gaussiano aditivo de sigma `noise_w` watts sobre la potencia LEIDA. La potencia verdadera se guarda
    aparte, solo para evaluar (el controlador nunca la ve).
  * `run()` registra UNA medicion por llamada, sin importar `duration` (hold/park no es una medicion nueva).
"""
import numpy as np


class StaticPlant:
    def __init__(self, curve, noise_w=0.0, rng=None):
        self.curve = curve
        self.noise_w = float(noise_w)
        self.rng = rng if rng is not None else np.random.default_rng(0)
        self.Ipv = 0.0
        self.log_I, self.log_Ptrue, self.log_Pmeas = [], [], []

    def set_curve(self, curve):
        self.curve = curve

    def p_true(self, I):
        """Potencia estatica verdadera en la corriente I (0 si I > Isc del patron)."""
        c = self.curve
        return float(np.interp(max(float(I), 0.0), c.I, c.P, right=0.0))

    def run(self, duration, I_ref):
        I = max(float(I_ref), 0.0)
        Pt = self.p_true(I)
        Pm = Pt + (self.rng.normal(0.0, self.noise_w) if self.noise_w > 0 else 0.0)
        self.Ipv = min(I, self.curve.Isc)                # corriente «medida»: no puede pasar de Isc
        self.log_I.append(I); self.log_Ptrue.append(Pt); self.log_Pmeas.append(Pm)
        return Pm, Pm * duration

    def reset_log(self):
        self.log_I, self.log_Ptrue, self.log_Pmeas = [], [], []


class DynamicPlantLogged:
    """Envoltura del simulador DINAMICO (converter_V4.Plant, nivel 2) con el mismo registro de mediciones que StaticPlant.
    La potencia &#x27;verdadera' es la que el panel entrega al FINAL de la ventana t_wait (arrastra la inercia del inductor);
    la &#x27;leida' es esa misma mas ruido."""

    def __init__(self, curve, wait, noise_w=0.0, rng=None, dt=1e-5):
        from pvsim_V4.converter_V4 import Plant
        self.plant = Plant(curve, dt=dt)
        self.curve, self.wait = curve, wait
        self.noise_w = float(noise_w)
        self.rng = rng if rng is not None else np.random.default_rng(0)
        self.log_I, self.log_Ptrue, self.log_Pmeas = [], [], []

    @property
    def Ipv(self):
        return self.plant.Ipv

    def set_curve(self, curve):
        self.curve = curve
        self.plant.set_curve(curve)

    def run(self, duration, I_ref):
        P, E = self.plant.run(self.wait if duration is None or duration <= 1.0 else duration, float(I_ref))
        Pm = P + (self.rng.normal(0.0, self.noise_w) if self.noise_w > 0 else 0.0)
        self.log_I.append(float(I_ref)); self.log_Ptrue.append(float(P)); self.log_Pmeas.append(float(Pm))
        return Pm, E

    def reset_log(self):
        self.log_I, self.log_Ptrue, self.log_Pmeas = [], [], []
```

pvsim_V4/gym_env_V4.py

60 LÍNEAS

```
"""
gym_env_V4 — Entorno Gymnasium de la V4: «proponer el maximo a partir de N potencias medidas» (propuesta del profesor).

Un EPISODIO = un patron de sombra desconocido (curva sintetica al azar, 4 sub-strings):
  1. El entorno mide N potencias en N corrientes uniformes y ascendentes (rango fijado con la Isc NOMINAL). Eso es la OBSERVACION
     (constante durante el episodio): vector de N potencias / 250 W.
  2. El agente propone una corriente (accion en [-1,1] -> [I_MIN, I_MAX]). Con politica gaussiana (PPO/SAC/REINFORCE) cada
     paso es una muestra de N(mu(obs), sigma^2): el «muestreo de puntos aledanos» es PARTE de policy gradients, no una etapa aparte.
  3. Se hacen K pasos (K muestras). Recompensa de cada paso = potencia MEDIDA en la corriente propuesta / 250 W.
Reglas anti-fuga (profesor): la recompensa y la observacion usan SOLO potencias medidas; el GMPP/P_ceiling NO se usan en
entrenamiento (solo en las metricas de evaluacion, que salen en `info[&#x27;metrics']` al terminar).
Costo en mediciones por episodio = N + K  (el MSAPSO del paper usa 175).
Como cada episodio es de un solo contexto con K muestras independientes, es un problema de bandido contextual: usar gamma = 0.
level="static": potencia estatica P(I) (nivel 1, Colab del profesor). level="dynamic": simulador con Boost + Super-Twisting (nivel 2).
"""
import numpy as np
import gymnasium as gym
```

```
from gymnasium import spaces

from pvsim_V4.array_model_V4 import SyntheticPVCurve, random_shading_mix
from pvsim_V4.static_plant_V4 import StaticPlant, DynamicPlantLogged
from pvsim_V4.medicion_V4 import grid_currents, action_to_current, metrics_from_plant, P_NOM


class MPPTSampleEnvV4(gym.Env):
    metadata = {"render_modes": []}

    def __init__(self, N=7, K=5, noise_w=0.0, level="static", wait=5e-4, n_sub=4):
        super().__init__()
        self.N, self.K, self.noise_w, self.level, self.wait, self.n_sub = N, K, noise_w, level, wait, n_sub
        self.action_space = spaces.Box(-1.0, 1.0, shape=(1,), dtype=np.float32)
        self.observation_space = spaces.Box(0.0, 10.0, shape=(N,), dtype=np.float32)
        self._rng = np.random.default_rng()

    def _plant(self, curve):
        if self.level == "static":
            return StaticPlant(curve, self.noise_w, rng=self._rng)
        return DynamicPlantLogged(curve, self.wait, self.noise_w, rng=self._rng)

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        if seed is not None:
            self._rng = np.random.default_rng(seed)
        options = options or {}
        self.curve = options.get("curve") or SyntheticPVCurve(random_shading_mix(self._rng, n_sub=self.n_sub))
        self.plant = self._plant(self.curve)
        P = np.array([self.plant.run(1.0, i)[0] for i in grid_currents(self.N)])        # barrido ascendente de N puntos
        self.obs = np.clip(P / P_NOM, 0.0, 10.0).astype(np.float32)
        self.k = 0
        return self.obs.copy(), {}

    def step(self, action):
        I = action_to_current(float(np.asarray(action).reshape(-1)[0]))
        Pm, _ = self.plant.run(1.0, I)
        self.k += 1
        done = self.k >= self.K
        info = {"I": I}
        if done:
            info["metrics"] = metrics_from_plant(self.plant, self.curve)
        return self.obs.copy(), float(Pm / P_NOM), done, False, info
```

reinforce_nn_V4.py

127 LÍNEAS

```
"""
reinforce_nn_V4.py — REINFORCE con MLP (propuesta del profesor) en el NIVEL 1 (potencia estatica).

Politica: MLP(N potencias/250 W) -> mu en [-1,1] (tanh) = corriente propuesta (accion normalizada). Se muestrean K acciones
a_k ~ N(mu, sigma^2) por patron de sombra (el «muestreo de puntos aledanos» ES la politica). Recompensa r_k = potencia MEDIDA
en la corriente muestreada / 250 W. NO se usa el GMPP ni P_ceiling en el entrenamiento (solo en la evaluacion).
Gradiente (REINFORCE):  grad J = E[ (r_k - b) * (a_k - mu)/sigma^2 * grad mu ].
Baselines b:  none | mean (promedio de los OTROS K-1 muestreos del mismo patron, «leave-one-out») | value (V(obs) aprendida).
sigma fija (en fraccion de la Isc nominal): «desviacion estandar muy pequenita» (profesor).
USO:  python3 reinforce_nn_V4.py --N 7 --K 5 --sigma 0.05 --baseline mean --seed 0
"""
import sys, os, json, time, argparse
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import numpy as np
import torch, torch.nn as nn
from pvsim_V4.array_model_V4 import SyntheticPVCurve, random_shading_mix
from pvsim_V4.static_plant_V4 import StaticPlant
from pvsim_V4.medicion_V4 import (ISC_NOM, I_MIN, I_MAX, P_NOM, grid_currents, scan, metrics_from_plant, current_to_action, action_to_current)

TABLE_PTS = 600


def p_table(curve):
    Ig = np.linspace(0.0, ISC_NOM, TABLE_PTS)
    return np.interp(Ig, curve.I, curve.P, right=0.0).astype(np.float32)


def build_pool(M, seed=0):
    rng = np.random.default_rng(seed)
    return np.stack([p_table(SyntheticPVCurve(random_shading_mix(rng))) for _ in range(M)])


def table_lookup(tab, I):
    """tab (B,T) ; I (B,K) amperes -> potencia (B,K) por interpolacion lineal."""
    pos = np.clip(I / ISC_NOM, 0.0, 1.0) * (TABLE_PTS - 1)
    lo = np.floor(pos).astype(int); hi = np.minimum(lo + 1, TABLE_PTS - 1); fr = pos - lo
    rows = np.arange(tab.shape[0])[:, None]
    return tab[rows, lo] * (1 - fr) + tab[rows, hi] * fr
```

```
class Policy(nn.Module):
    def __init__(self, N, hidden=64):
        super().__init__()
        self.mu = nn.Sequential(nn.Linear(N, hidden), nn.ReLU(), nn.Linear(hidden, hidden), nn.ReLU(), nn.Linear(hidden, 1), nn.Tanh())
        self.v = nn.Sequential(nn.Linear(N, hidden), nn.ReLU(), nn.Linear(hidden, 1))

    def forward(self, x):
        return self.mu(x).squeeze(-1)


def sigma_action(sigma_frac):
    return sigma_frac * ISC_NOM / (0.5 * (I_MAX - I_MIN))          # sigma en unidades de la accion [-1,1]


def train(N=7, K=5, sigma=0.05, baseline="mean", updates=3000, B=128, lr=1e-3, noise_w=0.0, seed=0, pool=None, log=True, adv_norm=True,
reward="abs"):
    torch.manual_seed(seed); rng = np.random.default_rng(seed)
    pool = pool if pool is not None else build_pool(6000, seed=0)
    Iscan = grid_currents(N)
    obs_all = np.stack([table_lookup(pool, np.tile(i, (pool.shape[0], 1)))[:, 0] for i in Iscan], axis=1) / P_NOM   # (M,N)
    pol = Policy(N); opt = torch.optim.Adam(pol.parameters(), lr=lr)
    sa = sigma_action(sigma); t0 = time.time(); hist = []
    for u in range(updates):
        idx = rng.integers(0, pool.shape[0], B)
        obs = obs_all[idx] + (rng.normal(0, noise_w, (B, N)) / P_NOM if noise_w > 0 else 0.0)
        x = torch.tensor(obs, dtype=torch.float32)
        mu = pol(x)                                                  # (B,)
        eps = torch.randn(B, K)
        a = mu[:, None].detach() + sa * eps                          # (B,K) acciones muestreadas
        I = I_MIN + 0.5 * (np.clip(a.numpy(), -1, 1) + 1.0) * (I_MAX - I_MIN)
        P = table_lookup(pool[idx], I) + (rng.normal(0, noise_w, (B, K)) if noise_w > 0 else 0.0)
        if reward == "gain":                                         # mejora sobre el mejor punto del barrido (potencias MEDIDAS)
            P = P / P_NOM - obs.max(axis=1, keepdims=True)
        else:
            P = P / P_NOM                                            # recompensa: SOLO potencia medida
        R = torch.tensor(P, dtype=torch.float32)
        if baseline == "none":
            adv = R
        elif baseline == "mean":
            adv = R - (R.sum(1, keepdim=True) - R) / (K - 1)
        else:
            v = pol.v(x).squeeze(-1); adv = R - v[:, None].detach()
        if adv_norm and baseline != "none":
            adv = adv / (adv.std() + 1e-6)
        logp_grad = (a - mu.detach()[:, None]) / sa ** 2              # d log N / d mu (coeficiente; el gradiente pasa por mu)
        loss = -(adv * logp_grad * mu[:, None]).mean()               # grad = adv * (a-mu)/sigma^2 * grad mu
        if baseline == "value":
            loss = loss + ((pol.v(x).squeeze(-1)[:, None] - R) ** 2).mean()
        opt.zero_grad(); loss.backward(); opt.step()
        if log and (u % 500 == 0 or u == updates - 1):
            hist.append((u, float(R.mean())))
            print(f"  upd {u:5d}  recompensa media {R.mean():.4f}  ({time.time()-t0:.0f} s)", flush=True)
    return pol, hist


def eval_policy(pol, N, K, sigma, curves, mode="stoch", noise_w=0.0, seed=11, reps=1):
    """mode &#x27;stoch': N + K muestras gaussianas.  mode 'mu': N + 1 medicion en mu (determinista).  mode 'mu+stoch': N + 1 + K."""
    sa = sigma_action(sigma); out = []
    for k, c in enumerate(curves):
        for r in range(reps):
            rng = np.random.default_rng(seed + 1000 * k + r)
            pl = StaticPlant(c, noise_w=noise_w, rng=rng)
            _, P = scan(pl, N)
            with torch.no_grad():
                mu = float(pol(torch.tensor(P / P_NOM, dtype=torch.float32)[None])[0])
            if mode in ("mu", "mu+stoch"):
                pl.run(1.0, action_to_current(mu))
            if mode in ("stoch", "mu+stoch"):
                for _ in range(K):
                    pl.run(1.0, action_to_current(mu + sa * rng.normal()))
            out.append(metrics_from_plant(pl, c))
    return {m: float(np.nanmean([o[m] for o in out])) for m in out[0]}


if __name__ == "__main__":
    from baselines_nivel1_V4 import scenarios
    ap = argparse.ArgumentParser()
    ap.add_argument("--N", type=int, default=7); ap.add_argument("--K", type=int, default=5)
    ap.add_argument("--sigma", type=float, default=0.05); ap.add_argument("--baseline", default="mean", choices=["none", "mean", "value"])
    ap.add_argument("--updates", type=int, default=3000); ap.add_argument("--seed", type=int, default=0)
    ap.add_argument("--noise", type=float, default=0.0); ap.add_argument("--lr", type=float, default=1e-3)
    a = ap.parse_args()
    real, synth = scenarios()
    pol, hist = train(a.N, a.K, a.sigma, a.baseline, a.updates, lr=a.lr, noise_w=a.noise, seed=a.seed)
    for mode in ("stoch", "mu", "mu+stoch"):
        r = eval_policy(pol, a.N, a.K, a.sigma, real, mode, a.noise, reps=20 if a.noise else 5)
        s = eval_policy(pol, a.N, a.K, a.sigma, synth, mode, a.noise, reps=3)
```

```
        print(f"{mode:9s} | reales: n={r[&#x27;n_meas']:.0f} fin {r['eff_final']:.2f} ener {r['eff_energy']:.2f} | sinteticos: n=
{s['n_meas']:.0f} fin {s['eff_final']:.2f} ener {s['eff_energy']:.2f}")
```

ppo_nivel1_V4.py

56 LÍNEAS

```
"""
ppo_nivel1_V4.py — PPO (stable-baselines3) sobre MPPTSampleEnvV4 en el NIVEL 1 (potencia estatica).
Bandido contextual: gamma = 0 (cada paso es una muestra independiente del mismo patron). MLP 64x64. sigma inicial pequena
(log_std_init = -2  =>  sigma = 0.135 en unidades de accion, ~0.057 de la Isc nominal); PPO aprende despues su propia sigma.
Recompensa = potencia MEDIDA / 250 W (sin GMPP). Evaluacion sobre reales (4) y sinteticos (100) nunca vistos.
USO:  python3 ppo_nivel1_V4.py --N 7 --K 5 --steps 200000 --seed 0
"""
import sys, os, json, time, argparse
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import numpy as np
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, VecMonitor
from pvsim_V4.gym_env_V4 import MPPTSampleEnvV4
from pvsim_V4.static_plant_V4 import StaticPlant
from pvsim_V4.medicion_V4 import scan, metrics_from_plant, action_to_current, P_NOM


def eval_sb3(model, N, K, curves, mode="stoch", noise_w=0.0, seed=11, reps=1):
    out = []
    for k, c in enumerate(curves):
        for r in range(reps):
            rng = np.random.default_rng(seed + 1000 * k + r)
            pl = StaticPlant(c, noise_w=noise_w, rng=rng)
            _, P = scan(pl, N)
            obs = np.clip(P / P_NOM, 0, 10).astype(np.float32)
            if mode == "mu":
                pl.run(1.0, action_to_current(model.predict(obs, deterministic=True)[0][0]))
            else:
                for _ in range(K):
                    pl.run(1.0, action_to_current(model.predict(obs, deterministic=False)[0][0]))
            out.append(metrics_from_plant(pl, c))
    return {m: float(np.nanmean([o[m] for o in out])) for m in out[0]}


if __name__ == "__main__":
    from baselines_nivel1_V4 import scenarios
    ap = argparse.ArgumentParser()
    ap.add_argument("--N", type=int, default=7); ap.add_argument("--K", type=int, default=5)
    ap.add_argument("--steps", type=int, default=200000); ap.add_argument("--seed", type=int, default=0)
    ap.add_argument("--noise", type=float, default=0.0); ap.add_argument("--log-std", type=float, default=-2.0)
    ap.add_argument("--tag", default=None); ap.add_argument("--ent", type=float, default=0.0)
    a = ap.parse_args()
    tag = a.tag or f"ppo_N{a.N}_K{a.K}_s{a.seed}"
    venv = VecMonitor(DummyVecEnv([lambda i=i: MPPTSampleEnvV4(N=a.N, K=a.K, noise_w=a.noise) for i in range(8)]))
    venv.seed(a.seed)
    model = PPO("MlpPolicy", venv, seed=a.seed, gamma=0.0, gae_lambda=0.0, n_steps=256, batch_size=256, verbose=0, ent_coef=a.ent,
                policy_kwargs=dict(net_arch=[64, 64], log_std_init=a.log_std))
    t0 = time.time(); model.learn(a.steps)
    os.makedirs("modelos_V4", exist_ok=True); model.save(f"modelos_V4/{tag}")
    real, synth = scenarios()
    res = dict(tag=tag, steps=a.steps, seconds=time.time() - t0, sigma_aprendida=float(np.exp(model.policy.log_std.detach().numpy()).mean()))
    for mode in ("stoch", "mu"):
        r = eval_sb3(model, a.N, a.K, real, mode, a.noise, reps=5); s = eval_sb3(model, a.N, a.K, synth, mode, a.noise, reps=3)
        res[mode] = dict(real=r, synth=s)
        print(f"{tag} {mode:5s} | reales: n={r[&#x27;n_meas']:.0f} fin {r['eff_final']:.2f} ener {r['eff_energy']:.2f} | sint: n=
{s['n_meas']:.0f} fin {s['eff_final']:.2f} ener {s['eff_energy']:.2f} | sigma={res['sigma_aprendida']:.3f} | {res['seconds']:.0f} s",
flush=True)
    json.dump(res, open(f"resultados_V4/{tag}.json", "w"), indent=1)
```

baselines_nivel1_V4.py

93 LÍNEAS

```
"""
baselines_nivel1_V4.py — Referencias (sin aprendizaje) en el NIVEL 1 (potencia estatica, sin dinamica del convertidor).
Cuenta MEDICIONES (iteraciones), la metrica prioritaria del profesor, y la energia en unidades de iteracion.

Metodos:
  MSAPSO fiel (Isc real)   Fig. 6 del paper: 7 particulas, beta 0.1->0.95, t_max=25 (175 mediciones). Semilla con la Isc REAL del patron.
  MSAPSO fiel (Isc nom.)   igual, pero sembrado con la Isc NOMINAL (como el rango del profesor) => comparacion justa con la V4.
  regla simple (nom.)      beta=0.95 fijo, t_max=5 (35 mediciones). [POST-HOC en la V2]
  mejor-de-N               N puntos uniformes y se elige el mejor (sin refinar): cota inferior sin aprendizaje.
  mejor-de-N + gauss.      mejor-de-N, luego K muestras N(mejor, sigma^2) y se elige el mejor: lo que haria la politica
                           SIN haber aprendido nada (control del aporte real del modelo).
USO: python3 baselines_nivel1_V4.py
"""
```

```python
import sys, os, json
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import numpy as np
from pvsim_V4.pvcurve_V4 import load_case
from pvsim_V4.array_model_V4 import SyntheticPVCurve, random_shading_mix
from pvsim_V4.msapso_V4 import Swarm
from pvsim_V4.static_plant_V4 import StaticPlant
from pvsim_V4.medicion_V4 import (ISC_NOM, I_MIN, I_MAX, W_ENERGIA, scan, metrics_from_plant)


def run_msapso(plant, curve, isc_seed, beta_fn, t_max):
    sw = Swarm(first_pass="ascending")
    sw.restart(isc_seed, first=True)
    for _ in range(t_max):
        sw.sweep(plant, 1.0)
        sw.update(beta_fn(sw.t))


def paper_beta(t, t_max=25, bmin=0.1, bmax=0.95):
    return bmin + (t / t_max) * (bmax - bmin)             # Eq. (14)


def m_msapso_real(plant, curve, rng):  run_msapso(plant, curve, curve.Isc, paper_beta, 25)
def m_msapso_nom(plant, curve, rng):   run_msapso(plant, curve, ISC_NOM, paper_beta, 25)
def m_regla_simple(plant, curve, rng): run_msapso(plant, curve, ISC_NOM, lambda t: 0.95, 5)


def make_best_of_n(N, K=0, sigma_frac=0.05):
    def f(plant, curve, rng):
        I, P = scan(plant, N)
        mu = I[int(np.argmax(P))]
        for _ in range(K):
            plant.run(1.0, float(np.clip(rng.normal(mu, sigma_frac * ISC_NOM), I_MIN, I_MAX)))
    return f


def scenarios(n_synth=100, seed=2026):
    rng = np.random.default_rng(seed)
    real = [load_case(i) for i in range(4)]
    synth = [SyntheticPVCurve(random_shading_mix(rng)) for _ in range(n_synth)]
    return real, synth


def evaluate(method, curves, noise_w=0.0, seed=7, reps=1):
    out = []
    for k, c in enumerate(curves):
        for r in range(reps):
            rng = np.random.default_rng(seed + 1000 * k + r)
            pl = StaticPlant(c, noise_w=noise_w, rng=rng)
            method(pl, c, rng)
            out.append(metrics_from_plant(pl, c))
    return {k: float(np.nanmean([o[k] for o in out])) for k in out[0]} | {"n99_alcanzado": int(np.isfinite([o["n99"] for o in out]).sum()),
"n": len(out)}


METHODS = {
    "MSAPSO fiel (Isc real)": m_msapso_real,
    "MSAPSO fiel (Isc nominal)": m_msapso_nom,
    "regla simple b=0.95,t_max=5 (nom.)": m_regla_simple,
    "mejor-de-7": make_best_of_n(7),
    "mejor-de-7 + 5 gauss. (s=0.05)": make_best_of_n(7, 5, 0.05),
    "mejor-de-10 + 5 gauss. (s=0.05)": make_best_of_n(10, 5, 0.05),
    "mejor-de-20": make_best_of_n(20),
    "mejor-de-20 + 5 gauss. (s=0.05)": make_best_of_n(20, 5, 0.05),
}

if __name__ == "__main__":
    real, synth = scenarios()
    res, lines = {}, []
    def P(s=""):
        print(s); lines.append(s)
    for noise in (0.0, 1.0, 2.0):
        P(f"\n=== NIVEL 1 (estatico), ruido de medicion sigma = {noise:g} W | {W_ENERGIA} mediciones de ventana de energia ===")
        P(f"{&#x27;metodo':40s} | {'REALES (4): n_meas  n99  efi_fin  energia':44s} | {'SINTETICOS (100): n_meas  n99  efi_fin  energia'}")
        for name, m in METHODS.items():
            a = evaluate(m, real, noise, reps=20 if noise > 0 else 1)
            b = evaluate(m, synth, noise, reps=3 if noise > 0 else 1)
            res[f"{name}|{noise:g}"] = dict(real=a, synth=b)
            P(f"{name:40s} | {a[&#x27;n_meas']:6.0f} {a['n99']:6.1f} {a['eff_final']:7.2f} {a['eff_energy']:8.2f}      | {b['n_meas']:6.0f}
{b['n99']:6.1f} {b['eff_final']:7.2f} {b['eff_energy']:8.2f}")
    json.dump(res, open("resultados_V4/baselines_nivel1_V4.json", "w"), indent=1)
    open("resultados_V4/baselines_nivel1_V4.txt", "w").write("\n".join(lines))
```